

Android 数据存储：SQLite 与第三方替代库对比及选择

Android 数据存储：SQLite 与第三方替代库（简洁版）

一、SQLite 核心介绍

1. 是什么？

- 轻量级**嵌入式关系型数据库**（无需单独安装服务器），Android 系统内置，无需额外依赖。
- 支持标准 SQL 语法（增删改查、事务、索引等），数据存储在本地文件（.db）中，适用于本地结构化数据存储（如用户信息、历史记录、清单数据等）。

2. 核心特点

优点	缺点
内置无需额外集成	需手动写 SQL 语句，易出错
占用资源少、性能稳定	需手动管理数据库版本升级
支持事务（ACID 特性）	大量数据操作时代码繁琐
适配所有 Android 版本	无默认缓存机制，需手动优化

3. 简单使用示例（原生 API）

步骤 1：创建数据库帮助类（继承 SQLiteOpenHelper）

Code block

```
1 public class MyDbHelper extends SQLiteOpenHelper {
2     // 数据库名 + 版本号
3     private static final String DB_NAME = "MyDataBase.db";
4     private static final int DB_VERSION = 1;
5
6     // 表名 + 字段
7     public static final String TABLE_USER = "user";
8     public static final String COLUMN_ID = "id";
9     public static final String COLUMN_NAME = "name";
```

```

10
11     // 创建表的 SQL
12     private static final String CREATE_TABLE_USER = "CREATE TABLE " +
TABLE_USER + " (" +
13         COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
14         COLUMN_NAME + " TEXT NOT NULL)";
15
16     public MyDbHelper(Context context) {
17         super(context, DB_NAME, null, DB_VERSION);
18     }
19
20     @Override
21     public void onCreate(SQLiteDatabase db) {
22         db.execSQL(CREATE_TABLE_USER); // 执行建表 SQL
23     }
24
25     @Override
26     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
27         // 数据库版本升级时执行（如新增字段、修改表结构）
28         db.execSQL("DROP TABLE IF EXISTS " + TABLE_USER);
29         onCreate(db);
30     }
31 }

```

步骤 2：增删改查操作（通过 SQLiteDatabase）

Code block

```

1  // 1. 获取数据库实例
2  MyDbHelper dbHelper = new MyDbHelper(context);
3  SQLiteDatabase db = dbHelper.getWritableDatabase(); // 可写数据库
4
5  // 2. 插入数据
6  ContentValues values = new ContentValues();
7  values.put(MyDbHelper.COLUMN_NAME, "张三");
8  long userId = db.insert(MyDbHelper.TABLE_USER, null, values);
9
10 // 3. 查询数据
11 Cursor cursor = db.query(
12     MyDbHelper.TABLE_USER,
13     new String[]{MyDbHelper.COLUMN_ID, MyDbHelper.COLUMN_NAME},
14     null, null, null, null, null
15 );
16 while (cursor.moveToNext()) {
17     String name =
cursor.getString(cursor.getColumnIndexOrThrow(MyDbHelper.COLUMN_NAME));

```

```
18 }
19 cursor.close(); // 必须关闭 Cursor，避免内存泄漏
20
21 // 4. 关闭数据库
22 db.close();
23 dbHelper.close();
```

4. 适用场景

- 本地少量结构化数据存储（如 APP 配置、用户收藏、离线缓存）。
- 对 SQL 语法熟悉，需求简单，无需复杂功能。

二、主流第三方替代库（简化开发）

原生 SQLite 代码繁琐、易出错，第三方库通过**注解**、**链式调用**、**ORM（对象关系映射）** 简化操作，以下是最常用的 3 个库：

1. Room（Google 官方推荐）

核心定位

- Android Jetpack 组件库，基于 SQLite 的 ORM 框架，官方维护，兼容性最好。
- 无需手动写 SQL，通过注解将 Java/Kotlin 实体类与数据库表映射。

核心特点

- 优点：官方支持、零依赖、编译时校验 SQL 语法、支持 LiveData/Flow（响应式数据监听）、简化版本升级。
- 缺点：功能相对基础，复杂查询需手动写 SQL 注解。

简单使用示例

Code block

```
1 // 1. 定义实体类（对应数据库表）
2 @Entity(tableName = "user")
3 public class User {
4     @PrimaryKey(autoGenerate = true)
5     private long id;
6     @ColumnInfo(name = "name")
7     private String name;
8
9     // getter + setter
10 }
11
12 // 2. 定义 Dao 接口（数据操作方法）
```

```

13  @Dao
14  public interface UserDao {
15      @Insert
16      void insertUser(User user);
17
18      @Query("SELECT * FROM user")
19      List<User> getAllUsers();
20  }
21
22  // 3. 定义数据库实例
23  @Database(entities = {User.class}, version = 1)
24  public abstract class AppDatabase extends RoomDatabase {
25      public abstract UserDao userDao();
26
27      // 单例模式
28      private static AppDatabase instance;
29      public static synchronized AppDatabase getInstance(Context context) {
30          if (instance == null) {
31              instance = Room.databaseBuilder(
32                  context.getApplicationContext(),
33                  AppDatabase.class,
34                  "AppDb.db"
35              ).build();
36          }
37          return instance;
38      }
39  }
40
41  // 4. 调用操作
42  AppDatabase db = AppDatabase.getInstance(context);
43  UserDao userDao = db.userDao();
44
45  // 插入
46  userDao.insertUser(new User("李四"));
47
48  // 查询
49  List<User> users = userDao.getAllUsers();

```

2. GreenDAO（高效稳定）

核心定位

- 轻量级 ORM 框架，以**性能优异**著称（通过代码生成机制，比反射型 ORM 更快）。
- 支持复杂查询、事务、索引优化，适合中大型 APP。

核心特点

- 优点：速度快、内存占用低、支持链式查询、自动生成代码（无需手动写 Dao）。
- 缺点：需配置 Gradle 插件生成代码，学习成本略高于 Room。

简单使用示例（关键步骤）

1. 配置 Gradle 依赖，同步后生成代码。
2. 定义实体类：

Code block

```
1  @Entity
2  public class User {
3      @Id(autoincrement = true)
4      private Long id;
5      private String name;
6      // getter + setter
7  }
```

3. 初始化 GreenDAO 并操作：

Code block

```
1  // 初始化（通常在 Application 中）
2  DaoMaster.DevOpenHelper helper = new DaoMaster.DevOpenHelper(context,
    "GreenDao.db");
3  SQLiteDatabase db = helper.getWritableDatabase();
4  DaoMaster daoMaster = new DaoMaster(db);
5  DaoSession daoSession = daoMaster.newSession();
6  UserDao userDao = daoSession.getUserDao();
7
8  // 插入
9  userDao.insert(new User(null, "王五"));
10
11 // 查询（链式调用）
12 List<User> users = userDao.queryBuilder()
13     .where(UserDao.Properties.Name.eq("王五"))
14     .list();
```

3. LitePal（国产轻量）

核心定位

- 郭霖开发的 Android 数据库框架，以**极简 API**为特色，无需写实体类注解，无需配置复杂依赖。
- 适合快速开发、需求简单的 APP，学习成本最低。

核心特点

- 优点：API 简洁（一行代码完成增删改查）、无需生成代码、支持 XML 配置表结构。
- 缺点：功能相对简单，不支持响应式编程，大型项目使用较少。

简单使用示例

1. 配置 XML 表结构（assets/litepal.xml）：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <litepal>
3      <dbname value="LitePalDb" />
4      <version value="1" />
5      <list>
6          <mapping class="com.example.User" /> <!-- 实体类全路径 -->
7      </list>
8  </litepal>
```

2. 定义实体类（无需注解）：

Code block

```
1  public class User extends DataSupport { // 继承 DataSupport
2      private long id;
3      private String name;
4      // getter + setter
5  }
```

3. 操作数据库：

Code block

```
1  // 插入
2  User user = new User();
3  user.setName("赵六");
4  user.save();
5
6  // 查询
7  List<User> users = DataSupport.findAll(User.class);
```

三、库的选择建议

场景	推荐库
官方生态、Jetpack 项目、响应式需求	Room
中大型 APP、性能要求高、复杂查询	GreenDAO
小型 APP、快速开发、极简 API	LitePal
学习原生原理、需求极简单	原生 SQLite

总结：优先选 **Room**（官方维护、无兼容性问题），追求性能选 **GreenDAO**，快速开发选 **LitePal**，原生 SQLite 仅推荐用于学习或极简场景。

（注：文档部分内容可能由 AI 生成）